```
package com.polarbookshop.catalogservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication                                    ❶
public class CatalogServiceApplication {
  public static void main(String[] args) {                ❷
   SpringApplication.run(CatalogServiceApplication.class, args);
  }
}
```

❶ Defines a Spring configuration class and triggers component scanning and Spring Boot auto-configuration

❷ The method used to launch the application. It registers the current class to be run during the application's bootstrap phase.

The `@SpringBootApplication` annotation is a shortcut that includes three different annotations:

- `@Configuration` marks the class as a source of beans definitions.
- `@ComponentScan` enables component scanning to find and register beans in the Spring context automatically.
- `@EnableAutoConfiguration` enables the auto-configuration capabilities offered by Spring Boot.

Spring Boot auto-configuration is triggered by several conditions, such as the presence of certain classes in the classpath, the existence of specific beans, or the values of some properties. Since the Catalog Service project depends on `spring-boot-starter-web`, Spring Boot will initialize an embedded Tomcat server instance and apply the minimal configuration required to get a web application up and running in almost zero time.